



KeynetikPOS

Technical Manual

Architecture, build & developer guide

Version 2.0 • For software engineers
Qt 6 • C++17 • SQLite • CMake

Contents

1	Introduction & audience
2	Technology stack & dependencies
3	Repository layout
4	Building the project
5	Running & testing
6	Architecture overview
7	Ownership & dependency injection
8	The data layer: Money, Database, repositories
9	Schema & migrations
10	The sale pipeline
11	Accounting & ERP tier
12	UI architecture & theming
13	Security & licensing
14	Coding conventions
15	Extension recipes
16	Known gaps & technical debt

1. Introduction & audience

KeynetikPOS is a single-machine, offline-first point-of-sale and ERP application for the Kenyan retail market. This manual is for engineers maintaining or extending the codebase. It assumes working C++ and Qt knowledge and focuses on how the system is structured and why.

Companion documents: `CODE_DOCUMENTATION.md` (per-file reference) and the in-source header comments — every `.h/.cpp` opens with a WHAT / HOW / WHY block. This manual is the bird's-eye view those sit under.

2. Technology stack & dependencies

Concern	Choice
Language	C++17
UI / framework	Qt 6 (Widgets, Sql, Network, PrintSupport, SerialPort, Concurrent); Qt 5 fallback
Build	CMake $\geq$ 3.16, AUTOMOC/AUTOUIIC/AUTORCC
Database	SQLite via Qt's QSQLITE driver, WAL mode
Tests	QtTest (GUI-less), driven by ctest
Packaging	windeployqt + CPack/NSIS installer
Licence server	Cloudflare Worker + D1 (see license-server/)

There is no ORM and no third-party business-logic library — money, tax, payroll and the ledger are all implemented directly over Qt and SQLite.

3. Repository layout

The source tree is flat (one directory) — translation units sit at the repository root, tests under `tests/`. Files group into logical tiers:

Tier	Representative files
Domain / money math (dependency-free, unit-tested)	money.h, carttotals.*, salejournal.*, payroll.*, vat.*
Data layer	database.*, *repository.* (8 repos), ledger.*
Services	cartservice.*, checkoutservice.*, paymentmanager.*, discount.*, inventorymanager.*
UI	mainwindow.*, *dialog.*, cartmodel.*, productgridmodel.*, appstyle.*, colorscheme.*
Infrastructure	userManager.*, settingsmanager.*, secretstore.*, smtpclient.*, licensemanager.*, schedulemanager.*

4. Building the project

On Windows with the Qt MinGW kit (the verified configuration):

```
# add the Qt + MinGW toolchain to PATH, then build the Release preset
$env:PATH = "C:\Qt\Tools\mingw1310_64\bin;C:\Qt\6.11.0\mingw_64\bin;" + $env:PATH
cmake --build "build\Desktop_Qt_6_11_0_MinGW_64_bit-Release"
```

CMake presets (`CMakePresets.json` / `CMakeUserPresets.json`) define Qt-Debug / Qt-Release configurations that output under `out/`. An MSVC 2022 Debug configuration also exists under `build/`. The build is warning-clean under `-Wall -Wextra` (MSVC `/W4`); `-Werror` is not enabled.

All sources are listed explicitly

PROJECT_SOURCES in CMakeLists.txt enumerates every file (no globbing). When you add a .cpp/.h, add it there — and to the relevant test target if it is exercised by tests.

5. Running & testing

Six GUI-less QTest suites run under ctest (option KEYNETIK_BUILD_TESTS=ON, default):

Suite	Covers
carttotals_test	Subtotal / tax / discount / rounding (pure math).
database_test	recordSale + processRefund money/stock invariants and migration persistence, against an in-memory DB through the repositories.
ledger_test	Double-entry GL invariants.
payroll_test	Kenyan statutory deduction maths.
vat_test	VAT split + VAT-3 aggregation.
salejournal_test	Balanced journal-line construction for sales/purchases/expenses.

```
ctest --test-dir "build\Desktop_Qt_6_11_0_MinGW_64_bit-Release" --output-on-failure
```

CI (.github/workflows/ci.yml) builds and runs the suites on Ubuntu and does a full GUI build + ctest on Windows.

6. Architecture overview

KeynetikPOS layers cleanly: UI talks to services and the data facade; services hold business logic; the data facade delegates to per-aggregate repositories; pure-maths modules underpin money, tax and accounting and have no DB or UI dependency.

```
main() ■■■owns■■■> Database ■■■delegates■■■> SaleRepository / ProductRepository /
    ■ injects Database&                               CustomerRepository / SupplierRepository /
    v                                                    ExpenseRepository / PurchaseOrderRepository /
MainWindow(Database&)                                RefundRepository / SalesAnalyticsRepository
    ■ forwards Database&
    ■■■> dialogs (UI only) ■■■> CheckoutService ■■■> SaleRepository.recordSale (atomic)
    ■■■> managers (InventoryManager, ScheduleManager, ...) ■■■> Ledger (auto-post)
```

Pure maths (no DB/UI): Money, computeCartTotals, buildSaleJournal, Payroll, Vat

Design priorities

Money correctness (exact integer cents), transactional integrity at checkout, an auditable trail for every stock/discount/refund movement, and books that reconcile (a real double-entry ledger).

7. Ownership & dependency injection

There is no Database singleton. Exactly one Database is constructed in main(), owned for the process lifetime, and injected by reference:

```
// main.cpp
Database db;           // owns the one connection
db.initialize();
MainWindow w(db);      // injected down the tree
```

MainWindow holds a Database& m_db and forwards it to every dialog/manager/service it constructs; each consumer stores its own Database& m_db. This makes dependencies explicit, the data layer testable (tests own their own Database), and multiple connections possible. UserManager, LicenseManager and ThemeManager remain singletons by design.

8. The data layer: Money, Database, repositories

Money (money.h)

All monetary values are Money — a value type wrapping qint64 cents. Conversions are explicit (fromCents/fromMajor, cents()/toMajor()) so a stray double can never silently become money. Rounding happens once, at the UI/parse edge. Dates are likewise typed — QDateTime/QDate in structs, ISO strings only at the SQL boundary.

Database (database.h/.cpp) — the facade

Database owns the connection and the schema lifecycle, defines the domain structs (Product, Sale, SaleItem, SalePayment, SaleRequest, Supplier, PurchaseOrder, Expense, Customer), and exposes the full data API — but holds *no business SQL of its own*. Every data method delegates to a repository and propagates lastError() on writes:

```
int Database::recordSale(const SaleRequest &req) {
    SaleRepository repo(db);
    const int id = repo.recordSale(req);
    lastError = repo.lastError();
    return id;
}
```

The repository layer

Eight classes own the SQL for one aggregate each. Each takes a QSqlDatabase *by value* (a cheap ref-counted handle to the one open connection — same pattern as Ledger), so it shares the connection and any in-flight transaction without owning it.

Repository	Owns
SaleRepository	recordSale() + Sale/SaleItem queries
ProductRepository	catalog CRUD, stock mutators, adjustStockWithLog, history, valuation
CustomerRepository	customer CRUD, phone lookup, store-credit / loyalty
SupplierRepository	supplier CRUD (soft delete)
ExpenseRepository	expense categories + expenses + totals
PurchaseOrderRepository	PO CRUD + transactional receivePurchaseOrder
RefundRepository	processRefund / isRefunded
SalesAnalyticsRepository	takings, top sellers, payment totals, gross profit, P&L

Cross-aggregate operations compose on the shared connection: RefundRepository::processRefund reuses Sale + Product repositories, and PurchaseOrderRepository::receivePurchaseOrder reuses ProductRepository for the per-line audit log — all inside one transaction.

9. Schema & migrations

`initialize()` opens the DB (foreign keys + WAL + `busy_timeout`), creates tables idempotently, runs the versioned migrations, then the one-time money-to-cents conversion, and seeds sample data on a fresh DB.

- **Versioned migrations.** `runMigrations()` applies each numbered step newer than the `schema_meta` 'schema_version' row, each in its own transaction, written idempotently (via `ensureColumn`). A fresh DB and an old DB converge on the same shape.
- **Money-to-cents** is a one-time data conversion kept separate and gated by its own marker row — never by `schema_version`, since re-running would re-scale every amount by 100.

Adding a migration

Append a new entry to the migrations table in `runMigrations()` with the next version number and an idempotent body. Do not reuse or renumber existing versions.

10. The sale pipeline

Checkout is the system's most important transaction. The flow:

- 1 **CartService** holds the cart (multi-cart state machine); **CartModel** adapts it to the table view.
- 2 **computeCartTotals()** (`carttotals.h`, pure) derives subtotal/tax/discount/total in integer cents.
- 3 **CheckoutService::finalizeSale()** builds a `SaleRequest` (items, totals, tender, customer, cashier) and calls `Database::recordSale()`.
- 4 **SaleRepository::recordSale()** runs ONE transaction: re-validate stock, insert the sale, insert each line (snapshotting name/price/cost), decrement stock, accrue loyalty/credit, record split tenders. Any failure rolls back and returns -1.
- 5 On success the sale is auto-posted to the General Ledger (best-effort, isolated), the receipt is printed/emailed, inventory refreshes, and the action is audit-logged.

Stock is validated in the UI but ENFORCED in the transaction

The UI check is fast feedback; correctness comes from re-validating inside `recordSale`'s transaction, so two near-simultaneous sales cannot oversell. On failure the cart is preserved so the cashier can retry.

11. Accounting & ERP tier

A double-entry back office sits under the POS; all modules post into the same `QSqlDatabase` and use `Money`.

Module	Role
Ledger	Seeded Kenyan chart of accounts; <code>postEntry()</code> rejects unbalanced/empty entries; trial balance.
SaleJournal	Pure functions building balanced GL lines for sales / purchases / expenses; posted best-effort after the underlying record commits.
Payroll	NSSF/SHIF/Housing before PAYE; <code>computePayslip()</code> is pure + tested; <code>runPayroll</code> posts one balanced wage journal.
Vat / Etims	VAT-3 from the transactional tables (not the GL); per-product tax codes; eTIMS client interface + logging stub.

Auto-posting lives in the service/dialog layer (not in `database.cpp`) so the data tests stay isolated. A posting failure is logged but never undoes a durably recorded sale/purchase/expense.

12. UI architecture & theming

- **MainWindow** owns the subsystem objects and the dialogs; UI is built programmatically in `setupUI()` (no `.ui` files).

- **Model/View** — CartModel (QAbstractTableModel) and ProductGridModel (QAbstractListModel + filter proxy + painting delegate) back the cart table and product grid.
- **Theming** — ColorScheme is the palette; appstyle.cpp generates one app-wide stylesheet from it using dynamic-property selectors (kind/role/textScale/bold). ThemeManager is the single source of truth for light/dark; a toggle re-applies the sheet app-wide.
- **Dialogs** are UI-only and delegate to repositories/managers via the injected Database&.

13. Security & licensing

- **Passwords** — PBKDF2 (passwordhasher.h); legacy SHA-256 hashes are rehashed on next login. A forced-password-change flow covers the seeded admin.
- **Secrets at rest** — SecretStore uses Windows DPAPI (CryptProtectData) to encrypt the SMTP password; a 'dpapi:v1:' marker tags encrypted values and legacy plaintext is migrated on load.
- **Licensing** — LicenseManager activates online-first with a local fallback so offline shops are never locked out; the Cloudflare Worker + D1 server (license-server/) is the authority. Client-side key checks are a format gate, not security.
- **RBAC** — UserManager enforces role-based permissions; the UI disables disallowed actions and the rule is also checked in the business layer.

14. Coding conventions

- Every file opens with a WHAT / HOW / WHY header comment. Keep it current when behaviour changes.
- Classes PascalCase; functions camelCase; private members m_ prefixed.
- Money is always the Money type; dates are QDateTime/QDate in structs, ISO strings only at SQL bind.
- DB writes return bool/int with getLastError(); multi-part outcomes use result structs (e.g. CheckoutResult). Exceptions are not used (Qt-traditional).
- Repositories take QSqlDatabase by value; injected Database& is stored as m_db.
- Atomic operations wrap check + write in one transaction on the shared connection.

15. Extension recipes

Add a new repository method

- 1 Add the method to the relevant XRepository (.h + .cpp), using m_db and m_lastError.
- 2 Add a thin delegating method on Database that forwards and copies lastError() on writes.
- 3 If tests exercise it, the repo is already linked into database_test; add a case there.

Add a schema migration

- 1 Append { N+1, "desc", [this]{ return ensureColumn(...); } } to runMigrations().
- 2 Write the body idempotently; never renumber or reuse a version.
- 3 Add a database_test case asserting the new column/round-trip if it carries data.

Add a dialog

- 1 Give the constructor a Database& first parameter; store Database& m_db (declare it first, init first).
- 2 Build the UI in setupUI(); use property-based styling (kind/role/textScale) rather than inline colours.
- 3 Construct it from MainWindow passing m_db; add the files to PROJECT_SOURCES.

16. Known gaps & technical debt

Item	Notes
Shift not wired to checkout	ShiftManager / ZReportDialog exist but aren't on the live sale path; Sale.shiftId is forward-compatible plumbing (always 0 today).

Item	Notes
Error-handling conventions	Three styles coexist (bool+getLastError, int -1, result structs). A single Result would unify them.
Product.profitMargin is double	The one double in a domain struct; basis-points (int) would keep the struct float-free.
Inventory auto-refresh	InventoryManager polls on a timer; a single-process app could make this event-driven.
eTIMS	Only a logging stub; real KRA OSCU/VSCU integration needs device onboarding.

KeynetikPOS Technical Manual • Version 2.0 • Pairs with CODE_DOCUMENTATION.md and the in-source WHAT/HOW/WHY headers.